Praktikum Pemrograman Berbasis Web

Modul: Membangun Aplikasi Manajemen Buku dengan Laravel & Filament 3.x

---

📚 Daftar Isi

1. Pendahuluan
2. Filosofi Filament
3. Tujuan Pembelajaran
4. Prasyarat
5. Langkah-Langkah Praktikum
6. Tugas dan Evaluasi
7. Troubleshooting

📖 Pendahuluan

Dalam praktikum ini, mahasiswa akan membangun aplikasi **Sistem Manajemen Buku** menggunakan framework Laravel dan Filament 3.x. Aplikasi ini mencakup fitur lengkap CRUD (Create, Read, Update, Delete) untuk mengelola data buku dan penulis, dilengkapi dengan fitur pencarian, filter, upload gambar, dan sistem relasi database.

Waktu Pengerjaan

4-6 jam (dapat dibagi menjadi 2-3 sesi praktikum)

Tingkat Kesulitan

Menengah - Membutuhkan pemahaman dasar Laravel dan konsep MVC

🎯 Filosofi Filament

Apa itu Filament?

Filament adalah sebuah framework admin panel yang dibangun di atas Laravel dan Livewire. Filament dirancang untuk mempercepat pengembangan aplikasi web dengan menyediakan komponen-komponen siap pakai yang powerful dan elegan.

Mengapa Filament?

1. Rapid Application Development (RAD)

Filament memungkinkan developer untuk membangun admin panel yang kompleks dalam hitungan menit, bukan hari atau minggu. Dengan pendekatan declarative (deklaratif), Anda cukup mendefinisikan struktur data dan Filament akan menghasilkan UI yang lengkap.

```
// Hanya dengan beberapa baris kode, Anda mendapatkan:
// - Form yang indah
// - Tabel yang interaktif
// - Validasi otomatis
// - Responsive design
Forms\Components\TextInput::make('title')
    ->required()
    ->maxLength(255);
```

2. Full-Stack Components

Filament bukan hanya tentang UI. Ini adalah ekosistem lengkap yang mencakup:

- **Forms**: Komponen form yang kaya fitur dengan validasi built-in
- **Tables**: Tabel interaktif dengan sorting, filtering, searching, pagination
- **Actions**: Sistem aksi yang modular (edit, delete, bulk actions)
- **Notifications**: Sistem notifikasi yang elegan
- **Widgets**: Dashboard widgets untuk statistik dan visualisasi data
- **Infolists**: Tampilan read-only yang terstruktur untuk detail data

3. Convention over Configuration

Filament mengikuti prinsip "convention over configuration" - Anda mengikuti konvensi yang sudah ditetapkan, dan framework akan mengurus sisanya. Ini mengurangi boilerplate code dan mempercepat development.

4. Built on Modern Stack

- **Laravel**: Framework PHP modern dengan ekosistem yang matang
- **Livewire**: Membangun aplikasi dinamis tanpa menulis JavaScript
- **Alpine.js**: JavaScript ringan untuk interaktivitas
- **Tailwind CSS**: Utility-first CSS framework untuk styling yang konsisten

5. Developer Experience (DX)

Filament sangat memperhatikan pengalaman developer:

- **Intuitive API**: API yang mudah dipahami dan self-explanatory
- **Excellent Documentation**: Dokumentasi yang lengkap dan up-to-date
- **Type Safety**: Full support untuk PHP type hints dan IDE autocomplete
- **Extensible**: Mudah dikustomisasi dan diperluas sesuai kebutuhan

Filosofi Desain Filament

"Make the common things easy, and the hard things possible"

Filament dirancang dengan filosofi:

1. **Productive**: Fokus pada produktivitas developer
2. **Beautiful**: UI/UX yang modern dan menarik out-of-the-box
3. **Accessible**: Memenuhi standar accessibility (WCAG)
4. **Flexible**: Dapat dikustomisasi untuk berbagai use case
5. **Secure**: Security best practices built-in

Kapan Menggunakan Filament?

✅ **Gunakan Filament untuk:**

- Admin panel / CMS
- Dashboard internal perusahaan
- Aplikasi CRUD-heavy
- Backoffice applications
- Prototype dan MVP
- SaaS admin panels

❌ **Pertimbangkan alternatif untuk:**

- Public-facing websites (gunakan Livewire langsung atau Inertia.js)
- Aplikasi yang membutuhkan UI sangat custom

- Mobile apps (gunakan API + mobile framework)

Perbandingan dengan Alternatif Lain

Framework	Learning Curve	Customization	Speed	Best For
Filament	Medium	High	Very Fast	Admin panels, CRUD apps
Laravel Nova	Medium-High	Medium	Fast	Enterprise admin (berbayar)
Voyager	Low	Medium	Fast	Simple CMS
Custom Build	High	Unlimited	Slow	Unique requirements

🎓 Tujuan Pembelajaran

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan dapat:

1. ✅ Memahami konsep MVC (Model-View-Controller) dalam Laravel
2. ✅ Memahami filosofi dan kegunaan Filament dalam pengembangan web
3. ✅ Membuat dan mengelola database migrations
4. ✅ Membuat model dengan relasi (One-to-Many)
5. ✅ Mengimplementasikan CRUD menggunakan Filament Resources
6. ✅ Membuat form dengan validasi
7. ✅ Implementasi upload file dan storage management
8. ✅ Membuat fitur pencarian dan filtering
9. ✅ Menggunakan factory dan seeder untuk data dummy
10. ✅ Menerapkan authorization dengan Policies

📋 Prasyarat

Pengetahuan yang Diperlukan:

- ✅ Dasar-dasar PHP (OOP, namespace, traits)
- ✅ Pemahaman SQL dan database relasional
- ✅ Dasar-dasar HTML, CSS (Tailwind lebih baik)
- ✅ Familiar dengan command line / terminal
- ✅ Konsep MVC dan framework PHP (Laravel dasar)

Software yang Harus Terinstal:

- ✅ PHP >= 8.1
- ✅ Composer (dependency manager PHP)
- ✅ Database (SQLite/MySQL/PostgreSQL)
- ✅ Text Editor / IDE (VS Code, PHPStorm, dll)
- ✅ Git (untuk version control)
- ✅ Web Browser modern

Verifikasi Instalasi:

```
# Cek versi PHP
php -v
```

```
# Cek versi Composer
composer -V
```

```
# Cek versi Laravel (setelah install)
php artisan --version
```

🚀 Langkah-Langkah Praktikum

BAGIAN 1: Setup Project Laravel (30 menit)

Langkah 1.1: Membuat Project Laravel Baru

```
# Buat project Laravel baru
composer create-project laravel/laravel authbook
```

```
# Masuk ke direktori project
cd authbook
```

```
# Jalankan development server untuk test
php artisan serve
```

📝 Penjelasan:

- `composer create-project` membuat project Laravel baru dengan semua dependencies
- `authbook` adalah nama project (bisa diganti sesuai keinginan)
- Development server akan berjalan di `http://localhost:8000`

✅ **Checkpoint:** Buka browser dan akses `http://localhost:8000`, pastikan halaman welcome Laravel muncul.

Langkah 1.2: Konfigurasi Database

```
# Untuk SQLite (paling mudah untuk praktikum)
touch database/database.sqlite
```

Edit file `.env`:

```
DB_CONNECTION=sqlite
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=laravel
# DB_USERNAME=root
# DB_PASSWORD=
```

📝 Penjelasan:

- SQLite adalah database file-based, tidak perlu instalasi server terpisah
- File `database.sqlite` akan menyimpan semua data
- Comment (#) pada konfigurasi MySQL karena tidak dipakai

Alternatif menggunakan MySQL:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
```

```
DB_PORT=3306
DB_DATABASE=authbook
DB_USERNAME=root
DB_PASSWORD=
```

Langkah 1.3: Install Filament

```
# Install Filament via Composer
composer require filament/filament:"^3.0"
```

```
# Setup Filament panel
php artisan filament:install --panels
```

📝 Penjelasan:

- `filament/filament` adalah package utama Filament
- `--panels` membuat panel admin baru
- Akan membuat `AdminPanelProvider.php` secara otomatis

Input yang diminta:

- Panel ID: `admin` (tekan Enter untuk default)
- Filament akan publish assets dan setup konfigurasi

✅ **Checkpoint:** Cek apakah file `app/Providers/Filament/AdminPanelProvider.php` sudah ada.

BAGIAN 2: Membuat Model dan Database (45 menit)

Langkah 2.1: Membuat Model Author

```
# Buat model Author dengan migration
php artisan make:model Author -m
```

📝 Penjelasan:

- `-m` flag: sekaligus membuat file migration
- Model: representasi tabel dalam bentuk class PHP
- Migration: blueprint untuk struktur tabel database

Langkah 2.2: Edit Migration Author

Buka file `database/migrations/xxxx_xx_xx_create_authors_table.php`:

```
public function up(): void
{
    Schema::create('authors', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->timestamps();
    });
}
```

📝 Penjelasan:

- `id()`: Primary key auto-increment
- `string('name')`: Kolom VARCHAR untuk nama penulis
- `timestamps()`: Otomatis membuat `created_at` dan `updated_at`

Langkah 2.3: Edit Model Author

Buka file `app/Models/Author.php`:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Author extends Model
{
    use HasFactory;

    protected $fillable = [
```

```
        'name',
    ];

    public function books(): HasMany
    {
        return $this->hasMany(Book::class);
    }
}
```

📝 Penjelasan:

- `HasFactory`: Trait untuk factory (generate data dummy)
- `$fillable`: Kolom yang boleh diisi secara mass assignment
- `books()`: Relasi one-to-many ke model Book (1 author punya banyak books)

Langkah 2.4: Membuat Model Book

```
# Buat model Book dengan migration
php artisan make:model Book -m
```

Langkah 2.5: Edit Migration Book

Buka file `database/migrations/xxxx_xx_xx_create_books_table.php`:

```
public function up(): void
{
    Schema::create('books', function (Blueprint $table) {
        $table->id();
        $table->string('isbn')->unique();
        $table->string('title');
        $table->foreignId('author_id')->constrained()->onDelete('cascade');
        $table->string('publisher')->nullable();
        $table->year('year')->nullable();
        $table->string('cover')->nullable();
        $table->enum('status', ['available', 'borrowed'])->default('available');
        $table->timestamps();
    });
}
```

📝 Penjelasan Detail:

- `isbn`: International Standard Book Number (harus unique)
- `title`: Judul buku
- `foreignId('author_id')->constrained()`:
    - Membuat foreign key ke tabel `authors`
    - `constrained()` otomatis mendeteksi tabel tujuan dari nama kolom
    - `onDelete('cascade')`: Jika author dihapus, buku-bukunya ikut terhapus
- `nullable()`: Kolom boleh kosong
- `enum('status', [...])`: Kolom dengan nilai terbatas (available/borrowed)
- `default('available')`: Nilai default jika tidak diisi

Langkah 2.6: Edit Model Book

Buka file `app/Models/Book.php`:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Book extends Model
{
    use HasFactory;

    protected $fillable = [
        'isbn',
        'title',
        'author_id',
        'publisher',
        'year',
        'cover',
        'status',
    ];

    protected $casts = [
        'year' => 'integer',
    ];
```

```
    public function author(): BelongsTo
    {
        return $this->belongsTo(Author::class);
    }
}
```

📝 Penjelasan:

- `$casts`: Konversi tipe data otomatis (year ke integer)
- `author()`: Relasi many-to-one ke model Author (banyak books punya 1 author)

Langkah 2.7: Jalankan Migration

```
# Jalankan semua migration
php artisan migrate
```

✅ Checkpoint:

- Tidak ada error saat migration
- Cek database, tabel `authors` dan `books` sudah ada

```
# Lihat status migration
php artisan migrate:status
```

```
# Lihat struktur database
php artisan db:show
```

BAGIAN 3: Membuat Factory dan Seeder (30 menit)

Langkah 3.1: Membuat Factory untuk Author

```
php artisan make:factory AuthorFactory
```

Edit file `database/factories/AuthorFactory.php`:

```php
<?php

namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;

class AuthorFactory extends Factory
{
    public function definition(): array
    {
        return [
            'name' => fake()->name(),
        ];
    }
}
```

📝 Penjelasan:

- `fake()`: Helper Laravel untuk generate data palsu (menggunakan Faker library)
- `name()`: Generate nama acak yang realistis

Langkah 3.2: Membuat Factory untuk Book

```
php artisan make:factory BookFactory
```

Edit file `database/factories/BookFactory.php`:

```php
<?php

namespace Database\Factories;

use App\Models\Author;
use Illuminate\Database\Eloquent\Factories\Factory;

class BookFactory extends Factory
{
    public function definition(): array
    {
```

```
        return [
            'isbn' => fake()->unique()->isbn13(),
            'title' => fake()->sentence(3),
            'author_id' => Author::factory(),
            'publisher' => fake()->company(),
            'year' => fake()->numberBetween(1950, 2024),
            'status' => fake()->randomElement(['available', 'borrowed']),
        ];
    }
}
```

📝 Penjelasan:

- `unique()`: Pastikan nilai tidak duplikat
- `isbn13()`: Generate ISBN-13 format
- `sentence(3)`: Kalimat dengan 3 kata
- `Author::factory()`: Otomatis buat author baru jika diperlukan
- `company()`: Generate nama perusahaan
- `numberBetween()`: Angka random dalam range
- `randomElement()`: Pilih random dari array

Langkah 3.3: Edit Database Seeder

Edit file `database/seeders/DatabaseSeeder.php`:

```php
<?php

namespace Database\Seeders;

use App\Models\Author;
use App\Models\Book;
use App\Models\User;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        // Buat admin user untuk Filament
        User::factory()->create([
            'name' => 'Admin User',
```

```
            'email' => 'admin@example.com',
        ]);

        // Buat 10 authors
        $authors = Author::factory(10)->create();

        // Buat 50 books dengan random authors
        Book::factory(50)->create([
            'author_id' => fn() => $authors->random()->id,
        ]);
    }
}
```

📝 Penjelasan:

- `factory(10)->create()`: Buat 10 data sekaligus
- `fn() => $authors->random()->id`: Closure untuk pilih random author dari yang sudah dibuat
- Ini memastikan semua 50 buku terdistribusi ke 10 author yang ada

---

Langkah 3.4: Setup Storage Link

```
# Buat symbolic link untuk storage
php artisan storage:link
```

📝 Penjelasan:

- Membuat symlink dari `public/storage` ke `storage/app/public`
- Memungkinkan file upload (cover buku) bisa diakses via URL

Langkah 3.5: Jalankan Seeder

```
# Reset database dan seed data
php artisan migrate:fresh --seed
```

📝 Penjelasan:

- `migrate:fresh`: Drop semua tabel dan migrate ulang
- `--seed`: Sekaligus jalankan seeder

✅ **Checkpoint:** Verifikasi data terseed dengan benar:

```
php artisan tinker
```

Di dalam tinker:

```
App\Models\Author::count(); // Should return 10
App\Models\Book::count();   // Should return 50
exit;
```

BAGIAN 4: Membuat Filament Resources (60 menit)

Langkah 4.1: Membuat Author Resource

```
php artisan make:filament-resource Author --generate
```

📝 **Penjelasan:**

- `--generate`: Auto-generate form dan table berdasarkan struktur tabel
- Akan membuat file di `app/Filament/Resources/AuthorResource.php`

Langkah 4.2: Kustomisasi Author Resource

Edit file `app/Filament/Resources/AuthorResource.php`:

```
<?php

namespace App\Filament\Resources;

use App\Filament\Resources\AuthorResource\Pages;
use App\Models\Author;
use Filament\Forms;
```

```php
use Filament\Forms\Form;
use Filament\Resources\Resource;
use Filament\Tables;
use Filament\Tables\Table;

class AuthorResource extends Resource
{
    protected static ?string $model = Author::class;

    protected static ?string $navigationIcon = 'heroicon-o-user-group';

    protected static ?string $navigationGroup = 'Library Management';

    public static function form(Form $form): Form
    {
        return $form
            ->schema([
                Forms\Components\Section::make('Author Information')
                    ->schema([
                        Forms\Components\TextInput::make('name')
                            ->required()
                            ->maxLength(255)
                            ->placeholder('Enter author name'),
                    ]),
            ]);
    }

    public static function table(Table $table): Table
    {
        return $table
            ->columns([
                Tables\Columns\TextColumn::make('id')
                    ->sortable(),

                Tables\Columns\TextColumn::make('name')
                    ->searchable()
                    ->sortable(),

                Tables\Columns\TextColumn::make('books_count')
                    ->counts('books')
                    ->label('Books')
                    ->sortable()
                    ->alignCenter(),

                Tables\Columns\TextColumn::make('created_at')
                    ->dateTime()
                    ->sortable()
```

```
                    ->toggleable(isToggledHiddenByDefault: true),
            ])
            ->filters([
                //
            ])
            ->actions([
                Tables\Actions\EditAction::make(),
                Tables\Actions\DeleteAction::make(),
            ])
            ->bulkActions([
                Tables\Actions\BulkActionGroup::make([
                    Tables\Actions\DeleteBulkAction::make(),
                ]),
            ])
            ->defaultSort('name', 'asc');
    }

    public static function getRelations(): array
    {
        return [
            //
        ];
    }

    public static function getPages(): array
    {
        return [
            'index' => Pages\ListAuthors::route('/'),
            'create' => Pages\CreateAuthor::route('/create'),
            'edit' => Pages\EditAuthor::route('/{record}/edit'),
        ];
    }
}
```

📝 Penjelasan Komponen Form:

- `Section`: Mengelompokkan field dalam section
- `TextInput`: Input text biasa
- `required()`: Validasi wajib diisi
- `maxLength()`: Maksimal karakter
- `placeholder()`: Text hint di input

📝 Penjelasan Komponen Table:

- `TextColumn`: Menampilkan text

- `searchable()`: Bisa dicari
- `sortable()`: Bisa disorting
- `counts('books')`: Hitung relasi (aggregate query)
- `toggleable()`: Bisa show/hide kolom
- `defaultSort()`: Sorting default

Langkah 4.3: Membuat Book Resource

```
php artisan make:filament-resource Book --generate --view
```

📝 Penjelasan:

- `--view`: Sekaligus buat halaman view/detail

Langkah 4.4: Kustomisasi Book Resource

Edit file `app/Filament/Resources/BookResource.php`:

```php
<?php

namespace App\Filament\Resources;

use App\Filament\Resources\BookResource\Pages;
use App\Models\Book;
use Filament\Forms;
use Filament\Forms\Form;
use Filament\Resources\Resource;
use Filament\Tables;
use Filament\Tables\Table;

class BookResource extends Resource
{
    protected static ?string $model = Book::class;

    protected static ?string $navigationIcon = 'heroicon-o-book-open';

    protected static ?string $navigationGroup = 'Library Management';

    public static function form(Form $form): Form
    {
```

```
        return $form
            ->schema([
                Forms\Components\Section::make('Book Information')
                    ->schema([
                        Forms\Components\TextInput::make('isbn')
                            ->label('ISBN')
                            ->required()
                            ->unique(ignoreRecord: true)
                            ->maxLength(255),

                        Forms\Components\TextInput::make('title')
                            ->required()
                            ->maxLength(255),

                        Forms\Components\Select::make('author_id')
                            ->relationship('author', 'name')
                            ->searchable()
                            ->preload()
                            ->required()
                            ->createOptionForm([
                                Forms\Components\TextInput::make('name')
                                    ->required()
                                    ->maxLength(255),
                            ]),

                        Forms\Components\TextInput::make('publisher')
                            ->maxLength(255),

                        Forms\Components\TextInput::make('year')
                            ->numeric()
                            ->minValue(1000)
                            ->maxValue(date('Y'))
                            ->step(1),

                        Forms\Components\Select::make('status')
                            ->options([
                                'available' => 'Available',
                                'borrowed' => 'Borrowed',
                            ])
                            ->required()
                            ->default('available'),
                    ])
                    ->columns(2),

                Forms\Components\Section::make('Cover Image')
                    ->schema([
                        Forms\Components\FileUpload::make('cover')
```

```
                            ->image()
                            ->directory('book-covers')
                            ->visibility('public')
                            ->imageEditor()
                            ->maxSize(2048)
                            ->acceptedFileTypes(['image/jpeg', 'image/png',
'image/webp'])
                            ->helperText('Maximum file size: 2MB. Accepted formats:
JPEG, PNG, WebP'),
                    ]),
            ]);
    }

    public static function table(Table $table): Table
    {
        return $table
            ->columns([
                Tables\Columns\ImageColumn::make('cover')
                    ->circular()
                    ->defaultImageUrl(url('/images/default-book-cover.png')),

                Tables\Columns\TextColumn::make('isbn')
                    ->label('ISBN')
                    ->searchable()
                    ->sortable()
                    ->copyable(),

                Tables\Columns\TextColumn::make('title')
                    ->searchable()
                    ->sortable()
                    ->limit(50),

                Tables\Columns\TextColumn::make('author.name')
                    ->searchable()
                    ->sortable(),

                Tables\Columns\TextColumn::make('publisher')
                    ->searchable()
                    ->toggleable(isToggledHiddenByDefault: true),

                Tables\Columns\TextColumn::make('year')
                    ->sortable()
                    ->alignCenter(),

                Tables\Columns\BadgeColumn::make('status')
                    ->colors([
                        'success' => 'available',
```

```
                    'warning' => 'borrowed',
                ])
                ->icons([
                    'heroicon-o-check-circle' => 'available',
                    'heroicon-o-clock' => 'borrowed',
                ]),

            Tables\Columns\TextColumn::make('created_at')
                ->dateTime()
                ->sortable()
                ->toggleable(isToggledHiddenByDefault: true),
        ])
        ->filters([
            Tables\Filters\SelectFilter::make('status')
                ->options([
                    'available' => 'Available',
                    'borrowed' => 'Borrowed',
                ]),

            Tables\Filters\SelectFilter::make('year')
                ->options(function () {
                    $currentYear = date('Y');
                    $years = [];
                    for ($year = $currentYear; $year >= 1950; $year--) {
                        $years[$year] = $year;
                    }
                    return $years;
                })
                ->searchable(),

            Tables\Filters\SelectFilter::make('author')
                ->relationship('author', 'name')
                ->searchable()
                ->preload(),
        ])
        ->actions([
            Tables\Actions\ViewAction::make(),
            Tables\Actions\EditAction::make(),
            Tables\Actions\DeleteAction::make(),
        ])
        ->bulkActions([
            Tables\Actions\BulkActionGroup::make([
                Tables\Actions\DeleteBulkAction::make(),
            ]),
        ])
        ->defaultSort('created_at', 'desc');
}
```

```
    public static function getRelations(): array
    {
        return [
            //
        ];
    }

    public static function getPages(): array
    {
        return [
            'index' => Pages\ListBooks::route('/'),
            'create' => Pages\CreateBook::route('/create'),
            'view' => Pages\ViewBook::route('/{record}'),
            'edit' => Pages\EditBook::route('/{record}/edit'),
        ];
    }
}
```

📝 Penjelasan Komponen Form Lanjutan:

1. **Select dengan Relasi:**

```
Forms\Components\Select::make('author_id')
    ->relationship('author', 'name')  // Relasi model, kolom yang ditampilkan
    ->searchable()                    // Bisa dicari
    ->preload()                       // Load semua data di awal
    ->createOptionForm([...])         // Form untuk buat author baru inline
```

2. **File Upload:**

```
Forms\Components\FileUpload::make('cover')
    ->image()                         // Hanya terima gambar
    ->directory('book-covers')        // Simpan di folder ini
    ->visibility('public')            // Bisa diakses publik
    ->imageEditor()                   // Editor gambar built-in
    ->maxSize(2048)                   // Max 2MB (dalam KB)
```

3. **Badge Column:**

```
Tables\Columns\BadgeColumn::make('status')
    ->colors([...])                    // Warna berdasarkan nilai
    ->icons([...])                     // Icon berdasarkan nilai
```

4. **Filters:**

```
Tables\Filters\SelectFilter::make('status')
    ->options([...])                   // Opsi filter
```

Langkah 4.5: Kustomisasi View Book Page

Edit file `app/Filament/Resources/BookResource/Pages/ViewBook.php`:

```php
<?php

namespace App\Filament\Resources\BookResource\Pages;

use App\Filament\Resources\BookResource;
use Filament\Actions;
use Filament\Infolists;
use Filament\Infolists\Infolist;
use Filament\Resources\Pages\ViewRecord;

class ViewBook extends ViewRecord
{
    protected static string $resource = BookResource::class;

    protected function getHeaderActions(): array
    {
        return [
            Actions\EditAction::make(),
            Actions\DeleteAction::make(),
        ];
    }

    public function infolist(Infolist $infolist): Infolist
    {
        return $infolist
            ->schema([
                Infolists\Components\Section::make('Book Details')
```

```
                    ->schema([
                        Infolists\Components\ImageEntry::make('cover')
                            ->height(200)

->defaultImageUrl(url('/images/default-book-cover.png')),

                        Infolists\Components\TextEntry::make('isbn')
                            ->label('ISBN')
                            ->copyable(),

                        Infolists\Components\TextEntry::make('title')

->size(Infolists\Components\TextEntry\TextEntrySize::Large)
                            ->weight('bold'),

                        Infolists\Components\TextEntry::make('author.name')
                            ->label('Author'),

                        Infolists\Components\TextEntry::make('publisher'),

                        Infolists\Components\TextEntry::make('year'),

                        Infolists\Components\TextEntry::make('status')
                            ->badge()
                            ->color(fn (string $state): string => match ($state) {
                                'available' => 'success',
                                'borrowed' => 'warning',
                            }),

                        Infolists\Components\TextEntry::make('created_at')
                            ->dateTime(),

                        Infolists\Components\TextEntry::make('updated_at')
                            ->dateTime(),
                    ])
                    ->columns(2),
            ]);
    }
}
```

📝 Penjelasan Infolist:

- Infolist: Tampilan read-only untuk detail data
- Berbeda dengan form (untuk input), infolist hanya untuk menampilkan
- Support berbagai komponen (text, image, badge, dll)

BAGIAN 5: Authorization dengan Policies (30 menit)

Langkah 5.1: Membuat Policy untuk Book

```
php artisan make:policy BookPolicy --model=Book
```

Edit file `app/Policies/BookPolicy.php`:

```php
<?php

namespace App\Policies;

use App\Models\Book;
use App\Models\User;

class BookPolicy
{
    public function viewAny(User $user): bool
    {
        return true; // Semua user bisa lihat list
    }

    public function view(User $user, Book $book): bool
    {
        return true; // Semua user bisa lihat detail
    }

    public function create(User $user): bool
    {
        return true; // Semua user bisa buat buku baru
    }

    public function update(User $user, Book $book): bool
    {
        return true; // Semua user bisa edit
    }

    public function delete(User $user, Book $book): bool
    {
        return true; // Semua user bisa hapus
    }
```

```
    public function restore(User $user, Book $book): bool
    {
        return true;
    }

    public function forceDelete(User $user, Book $book): bool
    {
        return true;
    }
}
```

📝 Penjelasan Policy:

- Policy: Aturan authorization per model
- Setiap method return boolean (true = boleh, false = tidak boleh)
- Filament otomatis menggunakan policy jika ada
- Bisa dikustomisasi untuk role-based access

Contoh Role-Based:

```
public function create(User $user): bool
{
    return $user->role === 'admin' || $user->role === 'librarian';
}

public function delete(User $user, Book $book): bool
{
    return $user->role === 'admin'; // Hanya admin bisa hapus
}
```

Langkah 5.2: Membuat Policy untuk Author

```
php artisan make:policy AuthorPolicy --model=Author
```

Edit file `app/Policies/AuthorPolicy.php` (sama seperti BookPolicy, semua return `true`).

BAGIAN 6: Membuat Admin User dan Testing (30 menit)

Langkah 6.1: Membuat Admin User

```
php artisan make:filament-user
```

Input yang diminta:

- Name: `Admin`
- Email: `admin@example.com`
- Password: `password` (atau sesuai keinginan)

✅ **Checkpoint:** User admin berhasil dibuat

Langkah 6.2: Akses Admin Panel

```
# Jalankan development server
php artisan serve
```

1. Buka browser: `http://localhost:8000/admin/login`
2. Login dengan credentials yang baru dibuat
3. Explore fitur-fitur yang ada

Langkah 6.3: Testing CRUD Operations

Test Create Book:

1. Klik menu "Books" → "New Book"
2. Isi form:
    - ISBN: `978-0-545-01022-1`
    - Title: `Harry Potter and the Deathly Hallows`
    - Author: Pilih dari dropdown atau buat baru
    - Publisher: `Scholastic`
    - Year: `2007`
    - Status: `Available`
    - Upload cover image (opsional)

3. Klik "Create"

Test Search:

1. Di halaman list books
2. Ketik di search box: `Harry Potter`
3. Pastikan hasil muncul

Test Filter:

1. Klik icon filter
2. Filter by Status: `Available`
3. Filter by Year: `2007`
4. Pastikan hasil terfilter

Test Edit:

1. Klik icon edit pada buku
2. Ubah beberapa field
3. Klik "Save changes"

Test Delete:

1. Klik icon delete
2. Confirm deletion
3. Pastikan buku terhapus

Test Bulk Delete:

1. Checklist beberapa buku
2. Klik bulk action "Delete"
3. Confirm
4. Pastikan semua terhapus

Test View:

1. Klik icon view pada buku
2. Pastikan semua detail tampil dengan baik
3. Test edit/delete dari halaman view

BAGIAN 7: Verifikasi dan Optimasi (15 menit)

Langkah 7.1: Verifikasi Routes

```
php artisan route:list --path=admin
```

✅ **Checkpoint:** Pastikan semua routes Filament terdaftar

Langkah 7.2: Verifikasi Database

```
php artisan db:show
php artisan tinker
```

Di tinker:

```
# Import Models terlebih dahulu
>>> use App\Models\Author;
>>> use App\Models\Book;
```

```
// Cek jumlah data
Author::count();
Book::count();
```

```
// Cek relasi
$author = Author::first();
$author->books; // Tampilkan semua buku dari author ini

$book = Book::first();
$book->author; // Tampilkan author dari buku ini

exit;
```

Langkah 7.3: Clear Cache (Jika ada masalah)

```
php artisan optimize:clear
```

📝 Tugas dan Evaluasi

Tugas 1: Implementasi Dasar (Nilai: 40)

Implementasikan semua langkah di atas dan buktikan dengan:

- ✅ Screenshot admin panel yang berhasil login
- ✅ Screenshot list books dengan data yang terseed
- ✅ Screenshot form create book
- ✅ Screenshot hasil pencarian
- ✅ Screenshot filter aktif

Tugas 2: Kustomisasi dan Pengembangan (Nilai: 40)

Tambahkan fitur-fitur berikut:

1. **Tambah Field Description pada Book (10 poin)**
   - Migration: tambah kolom `description` (text)
   - Model: tambah ke `$fillable`
   - Resource: tambah `Textarea` component di form
   - Factory: generate description dengan `fake()->paragraph()`
2. **Implementasi Role-Based Access (15 poin)**
   - Tambah kolom `role` pada tabel users
   - Buat seeder untuk user dengan role berbeda (admin, staff, viewer)
   - Update BookPolicy:
     - Admin: semua akses
     - Staff: bisa create, edit, view (tidak bisa delete)
     - Viewer: hanya bisa view
3. **Tambah Widget Dashboard (10 poin)**
   - Buat widget untuk menampilkan:
     - Total books
     - Total authors
     - Books available vs borrowed (chart/stats)
4. **Tambah Kategori Buku (5 poin)**
   - Buat model Category
   - Relasi many-to-many dengan Book
   - Tambah di form book (multiple select)

Tugas 3: Analisis dan Dokumentasi (Nilai: 20)

Buat laporan yang berisi:

1. **Penjelasan Arsitektur (7 poin)**
    - Diagram ERD (Entity Relationship Diagram)
    - Penjelasan relasi antar tabel
    - Penjelasan flow aplikasi (dari request hingga response)
2. **Perbandingan dengan Pendekatan Tradisional (7 poin)**
    - Bandingkan membuat CRUD dengan Filament vs manual
    - Kelebihan dan kekurangan masing-masing
    - Kapan sebaiknya menggunakan Filament, kapan tidak
3. **Refleksi Pembelajaran (6 poin)**
    - Apa yang dipelajari dari praktikum ini?
    - Kesulitan yang dihadapi dan cara mengatasinya
    - Rencana pengembangan aplikasi selanjutnya

🔧 Troubleshooting

Problem 1: Error "Class 'App\Models\Book' not found"

Solusi:

```
composer dump-autoload
php artisan optimize:clear
```

Problem 2: Migration Error "Table already exists"

Solusi:

```
php artisan migrate:fresh --seed
```

Problem 3: Filament Page Not Found (404)

Solusi:

```
php artisan optimize:clear
php artisan route:clear
```

Problem 4: Upload Error "The file could not be uploaded"

Solusi:

```
php artisan storage:link
# Cek permission folder storage
chmod -R 775 storage
```

Problem 5: Seeder Error "Call to undefined method factory()"

Solusi:
Pastikan model menggunakan trait `HasFactory`:

```
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Book extends Model
{
    use HasFactory;
    // ...
}
```

Problem 6: Login Redirect Loop

Solusi:

```
# Clear semua cache
php artisan cache:clear
php artisan config:clear
php artisan view:clear
php artisan route:clear
```

📚 Referensi dan Sumber Belajar

Dokumentasi Resmi:

- Laravel Documentation
- Filament Documentation
- Livewire Documentation

Tutorial Video:

- Filament Official YouTube Channel
- Laravel Daily - Filament Tutorials

Community:

- Filament Discord
- Laravel Indonesia Telegram

🎯 Kriteria Penilaian

Aspek	Bobot	Kriteria
Implementasi Dasar	40%	- Semua fitur CRUD berfungsi - Database terisi dengan benar - UI responsif dan user-friendly
Kustomisasi	40%	- Fitur tambahan bekerja - Kode terstruktur dengan baik - Best practices diterapkan
Dokumentasi	20%	- Laporan lengkap dan jelas - Analisis mendalam - Refleksi bermakna

---

✅ Checklist Penyelesaian

Sebelum submit, pastikan semua ini sudah dicek:

- [ ] Project Laravel terinstall dengan benar
- [ ] Filament terinstall dan berjalan
- [ ] Database migration berhasil
- [ ] Model Author dan Book sudah ada dengan relasi yang benar
- [ ] Factory dan Seeder menghasilkan data dummy
- [ ] AuthorResource lengkap (form + table)
- [ ] BookResource lengkap (form + table + view + filters)
- [ ] File upload berfungsi untuk cover buku
- [ ] Search dan filter berfungsi
- [ ] Bulk delete berfungsi
- [ ] Policy sudah diterapkan
- [ ] Admin user bisa login
- [ ] Semua fitur sudah ditest manual
- [ ] Tugas tambahan (jika ada) sudah dikerjakan
- [ ] Dokumentasi/laporan sudah dibuat
- [ ] Code sudah di-commit ke Git repository

🎓 Kesimpulan

Praktikum ini memberikan pengalaman hands-on dalam:

1. ✅ Membangun aplikasi web modern dengan Laravel
2. ✅ Menggunakan Filament untuk rapid application development
3. ✅ Implementasi relasi database (one-to-many)
4. ✅ CRUD operations dengan best practices
5. ✅ Authorization dan security
6. ✅ File upload dan storage management

Filament membuktikan bahwa dengan tools yang tepat, developer dapat fokus pada business logic daripada boilerplate code, menghasilkan aplikasi yang powerful dalam waktu singkat.

Selamat belajar dan semoga sukses! 🚀

Dosen Pengampu:
Adi Wahyu Pribadi, S.Si., M.Kom

Mata Kuliah:
Pemrograman Berbasis Web

Semester/Tahun:
Gasal 2025-2026

© 2025 - Praktikum Pemrograman Berbasis Web